

Question No:9

DS	Session	Stack	Question Information	Level 1	Challenge 50
----	---------	-------	----------------------	---------	--------------

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
Create a string by concatenating the two operands and the operator after them.
string = operand1 + operand2 + operator
And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfix expressions

Output Format

Single line represents the prefix expression

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

ABC/-AK/L-*

EXPECTED OUTPUT

*-A/BC-/AKL

Test Case 2

INPUT (STDIN)

AB+CD-*

EXPECTED OUTPUT

*+AB-CD

▼ Mandatory Test Cases

Test Case 1

KEYWORD

string postToPre(string
post_exp)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

220

Test Case 3

NLOC

46

Code:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  #define MAX 100
6
7  char stack[MAX][MAX];
8  int top = -1;
9
10 void push(char str[])
11 {
12     top++;
13     strcpy(stack[top], str);
14 }
15
16 void pop(char str[])
17 {
18     strcpy(str, stack[top]);
19     top--;
20 }
21
22 int main()
23 {
24     char postfix[MAX];
25     char op1[MAX], op2[MAX];
26     char result[MAX];
27     char temp[2];
28     int i;
29
30     scanf("%s", postfix);
31
32     /* Scan postfix expression from left to right */
33     for (i = 0; postfix[i] != '\0'; i++)
34     {
35         /* If operand, push it */
36         if (isalnum((unsigned char)postfix[i]))
37         {
```

```

38     temp[0] = postfix[i];
39     temp[1] = '\\0';
40
41     push(temp);
42 }
43 else
44 {
45     /* Pop second operand first */
46     pop(op2);
47
48     /* Pop first operand */
49     pop(op1);
50
51     /*
52      Prefix = operator + operand1 + operand2
53     */
54     temp[0] = postfix[i];
55     temp[1] = '\\0';
56
57     strcpy(result, temp);
58     strcat(result, op1);
59     strcat(result, op2);
60
61     push(result);
62 }
63 }
64
65 printf("%s\\n", stack[top]);
66
67 return 0;
68

```

Output:

Test Case-1:

```
Output

ABC/ -AK/L - *
* -A/BC - /AKL

=== Code Execution Successful ===
```

Test Case-2:

```
Output

AB+CD - *
* +AB - CD

=== Code Execution Successful ===
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236